Mindscape
Research, build history and evidence review

Prepared for the project owner | 10 October 2026 | Detailed research and security report

Mindscape is an experimental architecture for explicit state, bounded actions, memory and verification around learned policies. It asks whether recording and controlling the process can improve learning, generalization and grounded execution. It is not a general-purpose foundation model or demonstrated multimodal intelligence.

Verified scope	State
Research v1 coding	1,920 learning cases; 400 lockbox episodes; 400 audits; 16 latency cases
Research v2 numeric	1,920 original interventions; 734 failures retained
Targeted scientific closure	288 new control executions; 85 failures retained; two live coding diagnostics
Full Research v2 release	Incomplete; no completion tag
Canonical repository	akilanpl/mindscape; inspected HEAD 33c5b56

This report separates observations, historical documentation and interpretation. Archived tests are reported as archived tests: no neural experiment or test suite was rerun merely to produce this document. Benchmark graphs were regenerated from saved machine-readable evidence. Historical external values were checked against the original reports.

Evidence: S01 README.md, S02 docs/final_research_report.md, S08 experiments/research_v2/release_manifest.json

How to read this package

The master report covers what the project is, the build sequence, tooling, each distinct benchmark, model performance, inference costs, observability, safety controls, scientific limitations and next research steps. The companion folder contains PDF editions of every Markdown document currently under docs/. They are preserved historical records, not all current statements of status.

Important terminology: a task is a distinct problem; an execution is one task under a particular seed and condition. A successful audit means a saved trajectory agrees with replay; it does not mean the model repaired the program. IID means sampled from the training structure family; OOD means held-out structures. A percentage point is the difference between two percentages, not a multiplicative speedup.

The name “Research v2” appears in older artifact directory names as well as the later scientific phase. The frozen coding package is treated here as Research v1, identified by its release tag. Directory naming alone does not establish the study or release identity.

Contents

Project definition and computation path

Build history and tools

Models and experimental regimes

Coding lockbox, learning curves and HumanEval

Earlier multiplication study and numeric interventions

Latency, acceleration and live telemetry

Security, safe development and remaining risks

Current status, priorities, reproduction and source ledger

No existing PDF files were found in the task workspace or supplied attachment directory at preparation time. The companion PDFs are newly generated editions of existing research Markdown. Their original text files, hashes and a document inventory are included. No external third-party technical report is represented as a Mindscape-authored PDF.

1. What Mindscape is and what we are doing

The central research hypothesis is that an explicit observation/state/action/event/result/goal representation may improve problem-budget efficiency, structural generalization and grounding compared with answer-oriented learning. The architecture makes execution inspectable: a model proposes an action, a controlled environment produces an actual result, the state changes, and an independent assessment checks the outcome.

The original domain is decimal multiplication. State explicitly records operands, digit positions, carries, partial rows, sign and goal. Later learned policies predicted numerical actions; a hand-designed cursor still organized the procedure. The coding domain replaces digits with repository files, symbols, relations, tests and typed edits. A secondary SQLite environment tests interface portability; its model study remains deferred.

Actual computation path

Input and task identity -> normalized observation -> structured state -> learned model request or deterministic selection -> bounded action -> environment result -> recorded state transition -> terminal verification -> final score.

In coding, the policy sees the broken repository, specification and visible tests. Training-only expert trajectories can populate verified memory. Private test inputs, expected values and reference patches stay outside evaluation-time observations. Terminal hidden tests run after policy termination. An asserted “success” from the model is insufficient.

Working memory resets for each episode; bounded episodic/concept memories retrieve prior executed training evidence. A static planner compares limited hypothetical branches. Predicted success is kept separate from real execution. Dreaming here is bounded hypothetical proposal generation, not a learned world model, consciousness, sleep or autonomous discovery.

The project contains useful interfaces and experimental controls, but its title and future placeholders should not be interpreted as completed multimodal capabilities. The evidence supports bounded mechanisms and narrow software-repair observations, not general intelligence.

Evidence: S01 README.md, S09 docs/coding/final_architecture.md, S10 docs/coding/protocol.md

2. How the build started and evolved

Stage / representative commit	What changed and what was learned
Foundation: d3d2261 -> 9151f1d	Specification, immutable schemas, decimal environment, replay verification and first validated deterministic milestone. An algorithmic ceiling is not learned reasoning.
Dataset infrastructure: 7a31943 -> 3a868e1	Versioned datasets, nested training budgets, split/disjointness checks and generic evaluation.
First learning: ef3d94e -> b94b39a	Trainable NumPy task/step policies, memory, checkpoints and preliminary multiseed curves. Early masking confounds emerged.
Controlled study: df98891 -> 6c8ea2f	Numerical claims, experiential feedback, fairness audits, failures and statistical analysis; primary OOD scores were zero.
Frozen feature study: 5cbda01 -> 7eeb73f	SmolLM2 feature backbone, stronger heads, controlled scoring, matched-information controls and preserved negative conclusions.
Coding vertical: a36f9b2 onward	WASI-isolated Python repair, Qwen backbones, real LoRA adaptation, lockbox, HumanEval, replay audits and packaging.
Completion / release: 2862f5a	CPU evidence preserved through measured MPS continuation, restarts and checkpoint recovery; all coding scope gates met.
Scientific v2: 0c3f869 -> 7cebc6d	Linux CPU numeric factorial, real traces, frozen tasks, failures and clean-clone restoration.
Gap closure: 06d1f8e -> c530faa	New zero-weight/masking controls, targeted task variants, direct coding model-forward observation and scoped validation.
Public presentation: 1a5efc4 -> 33c5b56	Six static pages and Vercel configuration. Presentation does not expand the scientific evidence.

These stages are reconstructed from Git history and preserved reports, not assigned invented elapsed dates. Historical Cloud recovery included inaccessible environments and an HTTP 403 push failure; later Linux analysis/validation did run. The completed coding sweep ultimately used local native MPS. Inaccessible processes were not treated as proven stopped, and completed CPU/checkpoint evidence was preserved.

Evidence: S01 README.md, S02 docs/final_research_report.md, S06 docs/research_v2/scientific_report.md, S07 docs/research_v2/gap_closure.md

3. Research tools and engineering approach

Tool / stack	Role and limits
Python, frozen dataclasses, TOML	Typed records, centralized domain transitions, configuration and offline core.
NumPy	Small learned policies and float64 CPU numeric intervention analysis.
PyTorch, Transformers, PEFT / LoRA	Pinned foundation inference and rank-eight adapter training; CPU float32 or measured MPS float16.
Hugging Face pinned snapshots	Exact backbone/tokenizer revisions and artifact hashes; pretraining exposure remains unknown.
Wasmtime + CPython WASI	Capability-limited execution of generated Python, independent tests and replay; not host Python execution.
SQLite	Bounded memory/storage and read-only SQL environment with authorizer and output/deadline limits.
unittest / pytest, lint, compile, wheel	Behavioral checks, statistical integrity, packaging and reproducibility. Historical pass counts belong to specific runs.
Git, GitHub Actions, SHA-256 archives	Prospective protocol freezing, versioned evidence, Linux runs, normal publication and clean restoration.
Native Apple M5 / MPS	Resident model, bounded batching and checkpointed inference; compute comparisons disclose precision changes.
HTML / CSS / JavaScript, Node, TypeScript	Recorded-evidence playback and static public site; no extra scientific result.
ReportLab, pypdf, Poppler	This evidence report, vector benchmark charts, PDF extraction and rendered layout checking.

The approach is incremental and skeptical: freeze a protocol, preserve task identities and source/model hashes, execute actual tasks, retain unsuccessful results, compare paired conditions when valid, audit raw outputs independently, restore into a clean destination, and state the boundary of each claim. Efficient continuation reuses completed keys and cached weights rather than rerunning completed sweeps.

Evidence: S01 README.md, S10 docs/coding/protocol.md, S16 docs/research_v2/reproduction.md

4. Models and conditions: do not mix studies

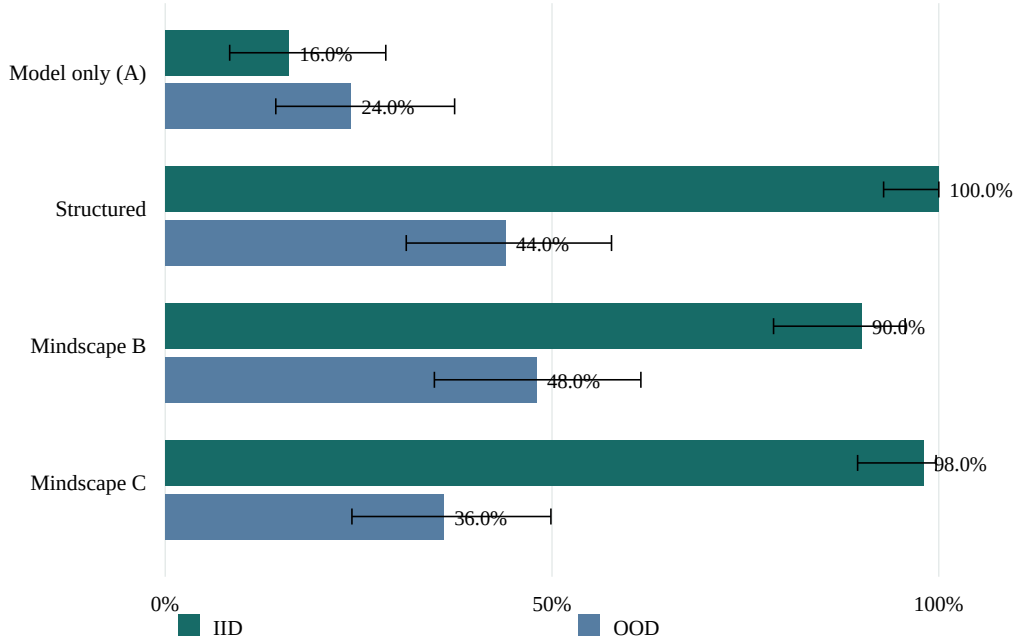
Model / regime	Concrete implementation
Historical NumPy MLP	Small task/action model. Controlled phase-4 models had 8,722 parameters; they are not LLMs.
SmolLM2-135M-Instruct	Frozen pretrained features plus matched 337,618-parameter trainable heads. 90 heads in the final main study; inference did not correct numerical claims.
Qwen2.5-Coder-0.5B-Instruct	494,032,768 parameters; preserved public HumanEval run, not the primary completed coding lockbox backbone.
Qwen2.5-Coder-1.5B-Instruct	1,543,714,304 parameters; revision 2e1fd397ee46e1388853d2af2c993145b0f1098a. Primary coding backbone.
LoRA adaptation	Rank eight on q_proj/v_proj; 1,089,536 trainable parameters. 61 preserved adapters were part of recovery inventories; older report counts cover different retained subsets.
A / structured coding	One source patch from plain or structured observation.
B / C coding	Typed multi-step paths, up to eight proposals and three accepted edits in the frozen completion comparison. Actual-experience replay is offline expert replay, not RL or self-play.
Research v2 numeric	Three preserved trained budget-50 seeds; float64 NumPy arithmetic. No text tokenizer. Zero-weight policies are copied intervention controls.

A, structured, B and C share a backbone inside each controlled study, but label content, prompt information, feedback and inference interactions differ. Comparing final percentages therefore cannot isolate the architecture wrapper. The earlier retrieval-budget study is also distinct from later gradient-trained LoRA budgets: memory retrieval examples are not optimizer training examples.

Evidence: S01 README.md, S09 docs/coding/final_architecture.md, S10 docs/coding/protocol.md, S03 experiments/coding_completion_v2/final_summary.json

5. Frozen coding lockbox: strongest narrow evidence

Coding lockbox: exact repair success



50 IID and 50 OOD tasks per condition; 400 condition/task executions over 100 independent tasks. Error bars are 95% Wilson intervals. Shared tasks are paired, not 400 independent problems.

Condition	IID success	OOD success	Interpretation
Model only	8/50 (16%)	12/50 (24%)	Plain adapted patch baseline
Structured	50/50 (100%)	22/50 (44%)	Highest IID ceiling; substantial OOD drop
B	45/50 (90%)	24/50 (48%)	Highest observed OOD point estimate
C	49/50 (98%)	18/50 (36%)	Strong IID, lower OOD than B/structured

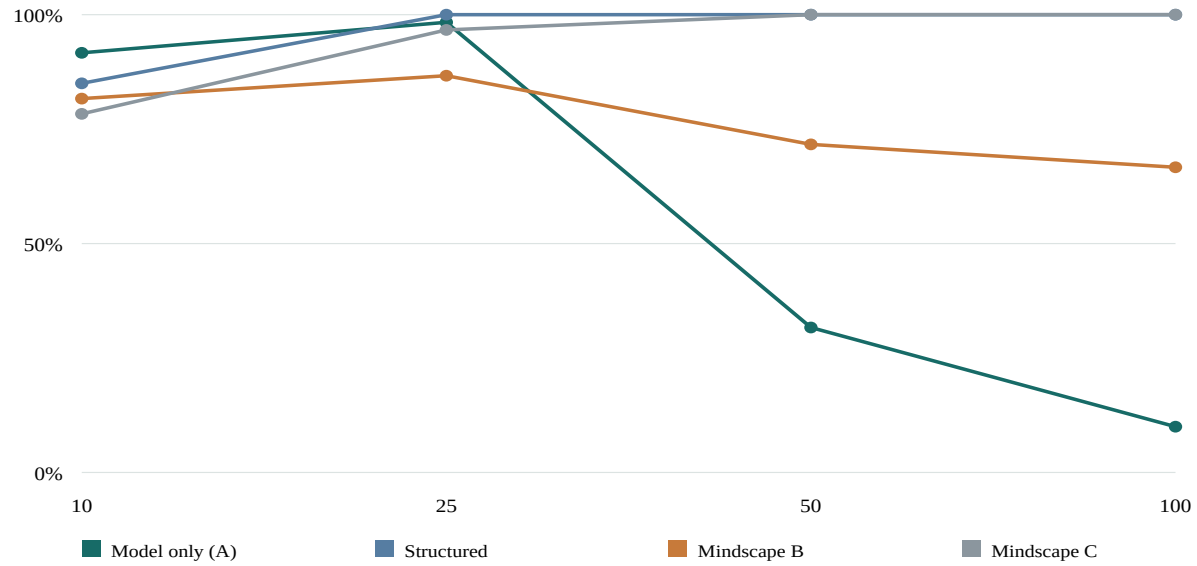
The conditions total 228 successful and 172 unsuccessful terminal repairs. All 400 audits agree with saved outcomes and state evidence. Replay agreement is an integrity result, not 100% repair accuracy. The five zero-tool trajectories have undefined tool-result agreement rather than a fabricated perfect per-episode score.

C is not uniformly better. Strong IID performance did not carry over to structural OOD. B has the highest OOD point estimate, but unequal supervision/interactions, shared task pairs and overlapping uncertainty prevent a simple architecture ranking. The completed study establishes coverage and observed behavior, not a universal advantage.

Evidence: S02 docs/final_research_report.md, S03 experiments/coding_completion_v2/final_summary.json

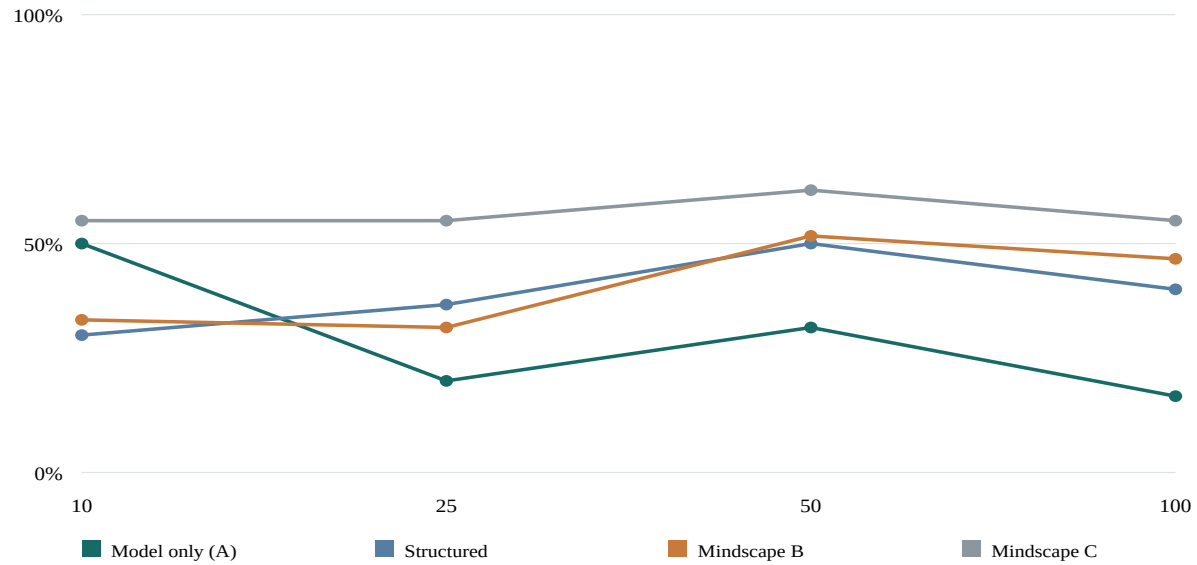
6. Coding learning curves and failure interpretation

Coding learning curves: IID



Each point combines three recorded learning seeds, 20 IID cases per seed (60 executions). Lines show observed aggregate rates, not population confidence intervals.

Coding learning curves: OOD



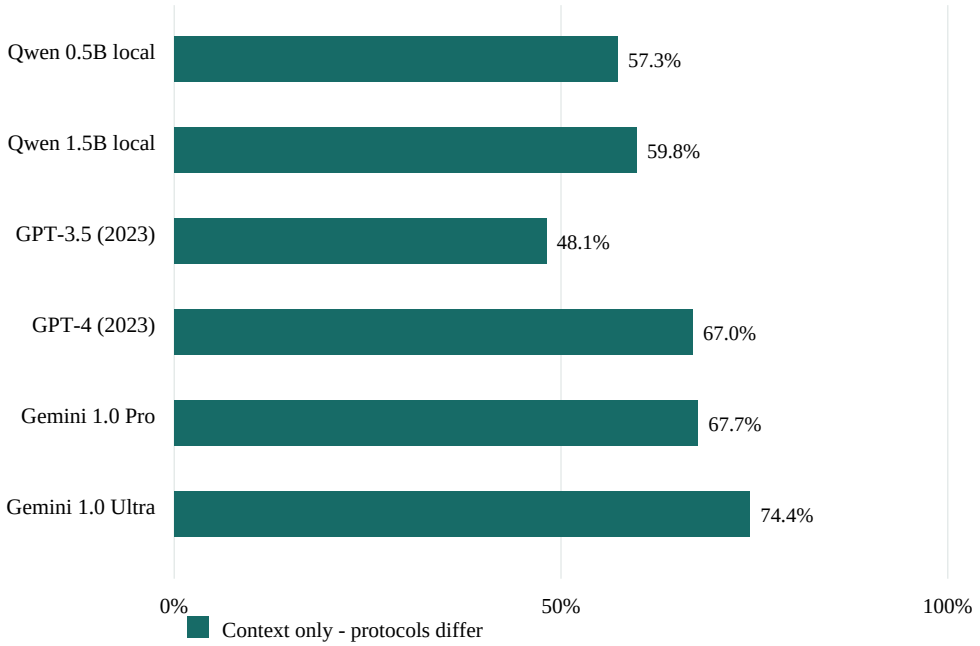
Same accounting for OOD. Budgets are 10, 25, 50 and 100 training tasks. CPU float32 and MPS float16 runs are mixed strata; the curves are descriptive, not a hardware-controlled causal estimate.

The completed matrix is four conditions x three seeds x four budgets x two splits x 20 cases = 1,920 executions. The original 1,122 CPU-row prefix and later 1,634-row restart prefix were preserved byte-for-byte. Increasing the task budget is not guaranteed to improve each cell. These repeated test sets and three seeds do not provide 1,920 independent samples.

Evidence: S03 experiments/coding_completion_v2/final_summary.json, S02 docs/final_research_report.md

7. Public HumanEval and older AI references

HumanEval: local measurements and historical reports



Local values: 94/164 and 98/164, one greedy completion, 512-token cap, CPython 3.14.7 WASI. GPT/Gemini bars are published historical values, not API reruns and not matched protocols. No common error bars are invented for the reports.

The 1.5B backbone achieved 59.76% locally, 2.44 percentage points above the 0.5B run. Against report values, its raw gap is -7.24 points to GPT-4, -7.94 to Gemini 1.0 Pro and -14.64 to Gemini 1.0 Ultra; it is +11.66 above reported GPT-3.5. These are numerical context gaps only. They do not establish significance, relative general intelligence or an architecture effect.

HumanEval here measures the foundation model completion protocol, not the custom interactive Mindscape repair lockbox. Comparing the custom 98% C IID result with GPT-4 HumanEval 67% would compare different tasks and be invalid. GPT/Gemini have not been evaluated on our frozen lockbox or controlled numeric task set. Those direct results remain unknown.

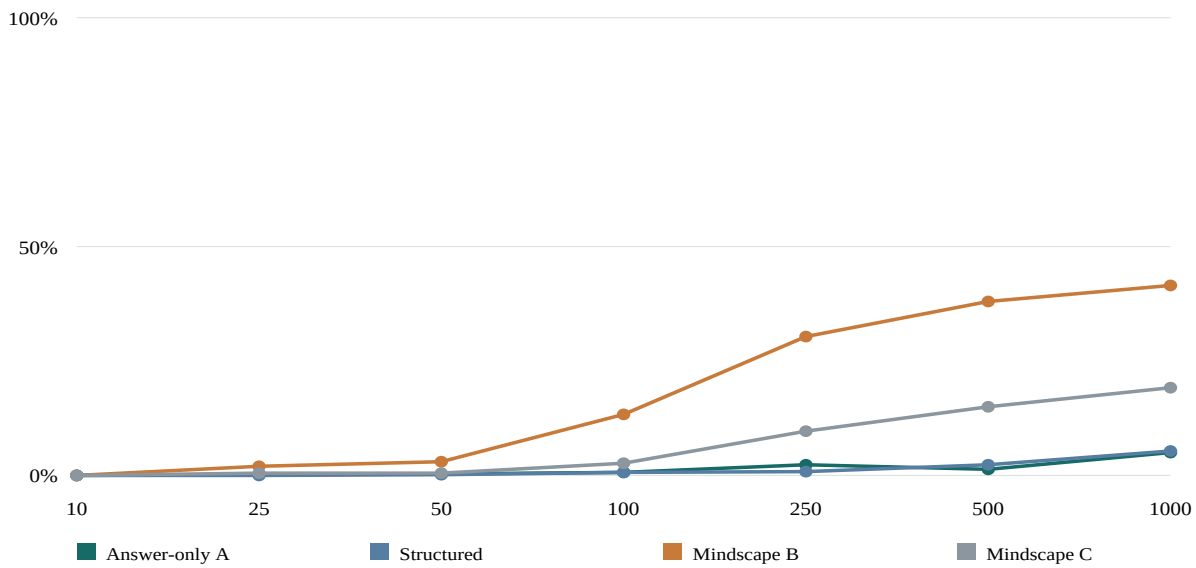
GPT-4 Table 2: GPT-3.5 48.1%, GPT-4 67.0% (March 2023 report). Gemini report v1 Table 2: Pro 67.7%, Ultra 74.4% (December 2023). Exact API checkpoints, complete decoding settings and contamination controls are not matched. No live paid API was called.

Evidence: S11 docs/coding/historical_gpt_gemini_comparison.md, S03 experiments/coding_completion_v2/final_summary.json

Primary external references: https://cdn.openai.com/papers/gpt-4.pdf ; https://arxiv.org/html/2312.11805v1 . Checked 10 October 2026.

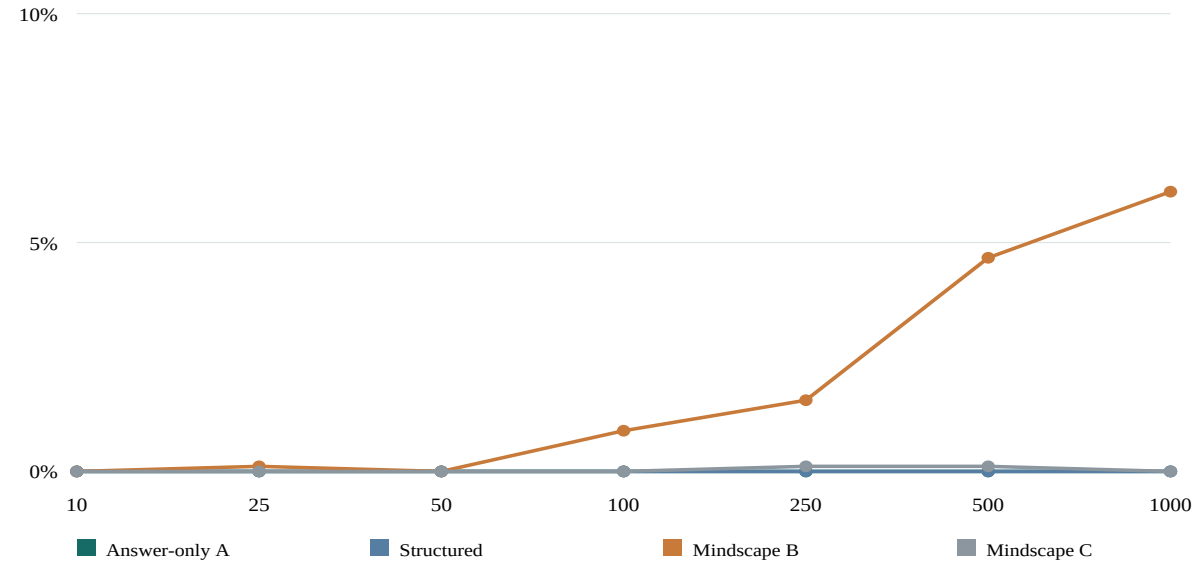
8. Earlier multiplication benchmarks

SmolLM2 frozen-feature study: IID



Seven nested budgets and three seeds. SmolLM2 features are frozen; lines are mean exact accuracy, not seed-SD intervals. IID has 200 cases per evaluation.

SmolLM2 frozen-feature study: OOD



OOD has 300 cases per evaluation. The vertical scale is 0-10%, unlike the IID plot. B reaches 6.111% at budget 1,000; most other OOD cells are zero.

At 1,000 examples: A IID 5.000% +/- 0.866 seed SD; structured 5.333% +/- 1.041; B 41.500% +/- 2.179; C 19.167% +/- 3.014. B OOD is 6.111% +/- 1.347. All observed 80/90/95% sample-efficiency thresholds were unreached; no 2x or 5x DER advantage is supported. Different supervision, cursor design and representation changes limit causal attribution.

Evidence: S04 docs/final_results.md, S05 experiments/final_smollm2_v1/aggregate.json

9. What the negative studies taught us

The original controlled numerical-claim MLP study measured B IID 6.333% and C IID 6.000% at 1,000 training problems, with zero exact accuracy on every primary OOD run. This did not reproduce the earlier exploratory high-OOD result. Failed evidence was preserved instead of discarded after a larger model improved some outcomes.

In the stronger frozen-feature study, the matched-information conventional local policy produced the same predictions as the B reference while bypassing its memory wrapper. A matched replay control also gave identical predictions. Therefore higher B accuracy over answer-only does not establish an independent wrapper or SQLite-storage advantage.

Dreaming changed no measured accuracy, groundedness or goal success at the tested budget. Its extra inference cost does not demonstrate usefulness. Removing state reduced some observed IID scores, but corrected per-seed evidence did not justify a universal contribution claim. Removing the whole memory/planner/verification system is not the same intervention as removing a feature readout.

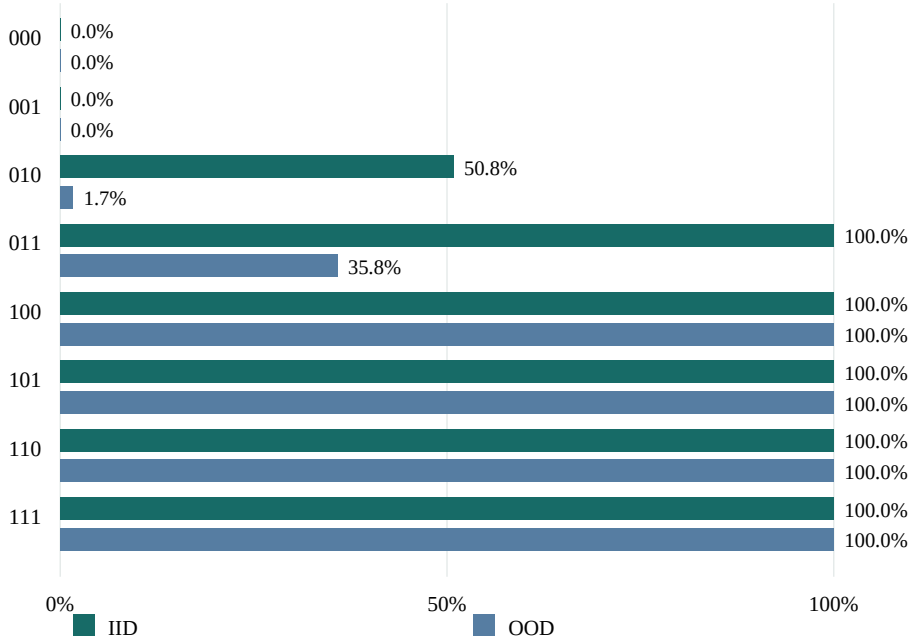
Failure taxonomy in 210 repeated-set evaluations included 22,367 arithmetic errors, 28,762 wrong actions, 133 unsupported answers and 1,238 correct outcomes. These are repeated-run counts, not independent unique errors. Cold-deployment efficiency and broad transfer were not established by cached evaluation timings.

Published GSM8K scores (GPT-4 92.0%, Gemini Ultra 94.4%) concern word problems and different prompting. They are not put on the multiplication graphs and cannot supply a meaningful Mindscape gap or training-data-efficiency ratio.

Evidence: S01 README.md, S04 docs/final_results.md, S12 docs/historical_references.md

10. Numeric v2: quantifying deterministic masking

Numeric factorial: success per 120 executions



Bit order is mask / state readout / previous-action readout. Each cell has 120 executions = 40 tasks x three seeds for the given split. The total is 80 distinct tasks x three seeds x eight conditions = 1,920 executions.

All 734 unsuccessful executions are retained. With both readouts present, disabling masking left IID at 120/120 but OOD fell from 120/120 to 43/120 (35.83%). The observed OOD effect is +64.17 percentage points, with task-cluster bootstrap 95% interval +50.0 to +77.5, conditional on the three shared trained checkpoints.

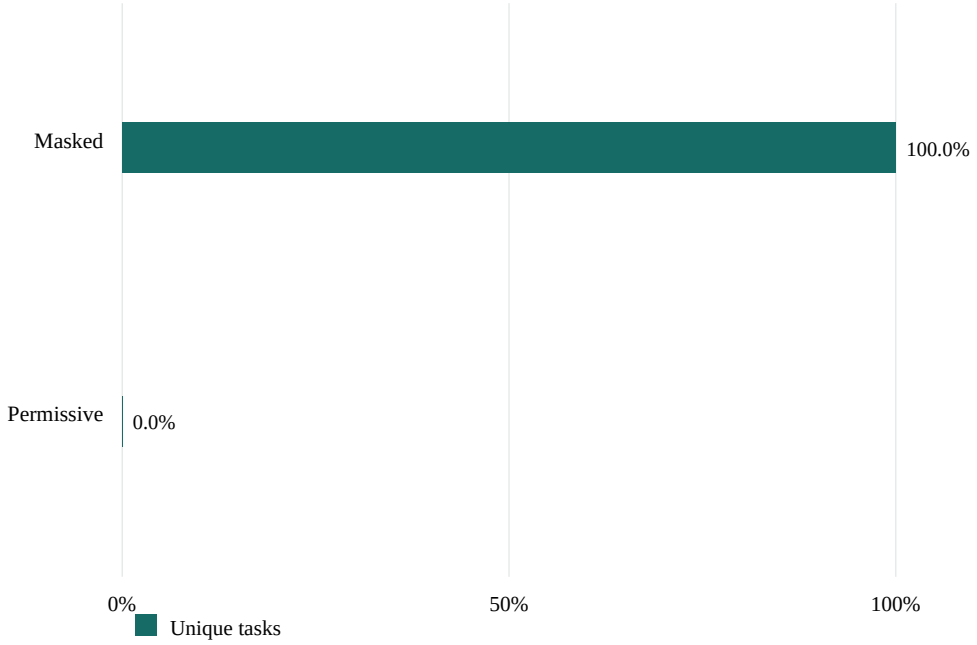
The direct cardinality audit found exactly one legal action at every one of 19,448 observed selections. The environment performs arithmetic; its singleton legal-action filter can select the next valid operation. Masked goal attainment therefore cannot be used as evidence that a neural network learned arithmetic or was needed for success.

This is a controlled software mechanism finding within the declared numeric environment. It is not a causal explanation of coding outcomes or a general theory of cognition. No text tokenizer, learned world model, planning, dreaming or feedback-learning component is present in this numeric backend.

Evidence: S06 docs/research_v2/scientific_report.md, S07 docs/research_v2/gap_closure.md

11. Targeted replication and zero-weight controls

Novel tasks: zero-weight controller comparison



24 unique, prospectively frozen tasks, disjoint from earlier identities. Masked zero-weight policies: 24/24; permissive zero-weight policies: 0/24. Identical zero-weight seed copies are not independent observations.

The observed paired effect is +100 percentage points. A post hoc conservative descriptive 95% interval is +66.6 to +100, derived from simultaneous Bonferroni Clopper-Pearson bounds on beneficial/harmful paired outcomes. This supplement does not replace the preregistered multiplicity-corrected tests.

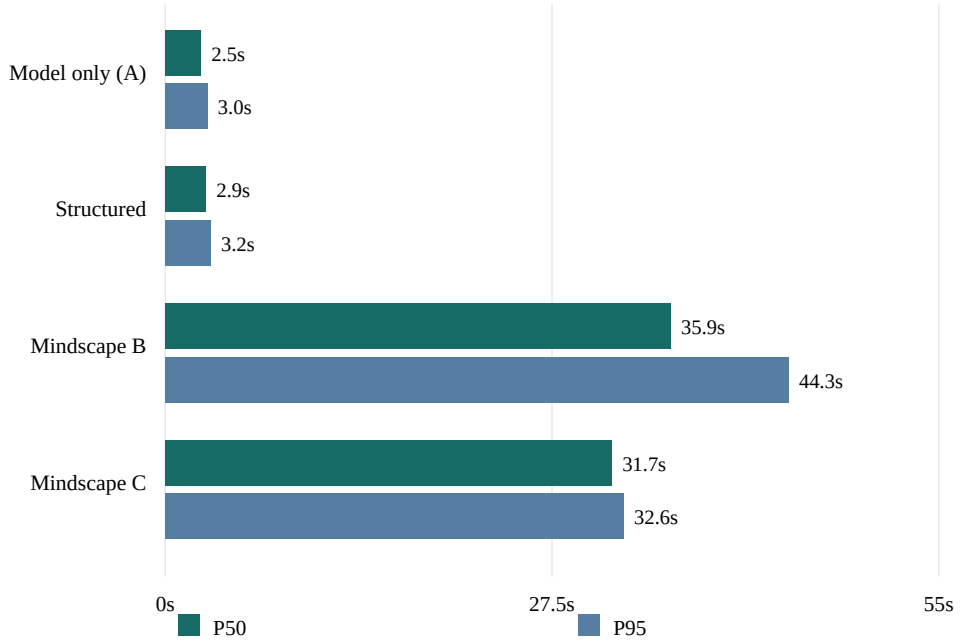
Novel family	Trained masked vs permissive	Effect / task-cluster 95% CI
Unsigned 3x2	24/24 vs 20/24	+16.7 pp; 0 to +41.7
Signed 4x2	24/24 vs 15/24	+37.5 pp; +8.3 to +66.7
Unsigned 1x4	24/24 vs 24/24	0 pp; observed bootstrap 0 to 0

Three historical trained seeds x 24 tasks x four conditions produced 288 executions, including 85 failures. Only eight unique tasks exist in each family. None of 18 predeclared per-seed tests passed Holm correction at 0.05 (minimum adjusted p = 0.140625). The qualitative finding replicated across new task variants, but not independent codebases, labs or training pipelines. A zero-width empirical interval at the ceiling does not establish population equivalence.

Evidence: S07 docs/research_v2/gap_closure.md, S08 experiments/research_v2/release_manifest.json

12. Performance, resources and acceleration

Sequential latency: four cases per condition



16 dedicated observations, four per condition. Sequential MPS float16, batch one, caching disabled, model resident. P95 from four samples is descriptive; these are not production latency guarantees.

The preserved acceleration receipt reports CPU 111.35 seconds per episode versus batched MPS amortized throughput 27.54 seconds per completed episode, a 4.04x ratio. Batch 16 and reduced precision changed together, so this is not isolated GPU-only speedup. Batch waiting time and individual episode wall latency are different metrics.

The later restart recovery averaged 24.79 seconds per episode after its diagnostic, as recorded in the completion history. It should not be substituted for the earlier pilot or the batch-one latency benchmark. Memory pressure, unused cache allocations and reboot downtime explain why whole-session elapsed time can differ from steady-state throughput.

Historical profiling reported 3.42 GB peak process RSS, 3.33 GB live MPS allocations and 12.53 GB driver memory. A later one-boundary release reduced unused driver allocations. Dedicated latency shared one process with about 3.80 GB process high-water RSS; per-row RSS is not an isolated per-task memory requirement. Linux numeric analysis used CPU and no purchased GPU. Dedicated cloud study peak memory and cost were not measured.

Neural tensors ran on MPS where receipt data established actual placement; filesystem operations, test subprocesses, checkpoints and Git remained CPU operations. Unsupported MPS operations were not silently reclassified as GPU success. No new profiling or inference was run for this report.

Evidence: S03 experiments/coding_completion_v2/final_summary.json, S04 docs/final_results.md, S02 docs/final_research_report.md

13. Observability: what we can actually inspect

The numeric study records input, normalization, task/checkpoint identities, feature slices, logits, derived probabilities, tensor shapes, action selection, memory changes, complete trajectory and independent outcome. Hidden summaries are explicitly reconstructed from observed features and weights; they are not mislabelled as stack-captured hidden activations.

A nonduplicating hash-bound index covers all 2,320 saved coding episodes (1,920 learning plus 400 lockbox). Historical prompts, token IDs and logits were not retained and cannot honestly be recovered as observed values. Two new observer diagnostics used the same historical task, cached Qwen 1.5B weights and original budget-100 adapters on CPU float32.

Diagnostic	Observed computation	Outcome / cost
Model only	1 model call; 39 direct token/logit forward pairs	Unsuccessful invalid edit envelope; 9.25 s including loading; peak RSS 8.46 GB
C	8 calls; 352 token/logit pairs; 8 retrieval, dreaming and planner invocations; 3 edits and memory updates	Final private evaluation passed; 5 later proposals hit edit budget; 71.98 s; peak RSS 7.16 GB

The initial observer attempt failed because PEFT generation bypassed an outer hook. Its failure log remains; the corrected observer captured the actual base-model forward path without extra inference. The diagnostics add zero accuracy benchmark cases and do not create a new speedup estimate.

The website explorer selects actual recorded events and labels selected versus total counts. Activation plots, logits and retrieved memories can show associations and execution, but cannot prove why an output occurred. Selected hidden activations and live telemetry coverage of all adapters remain unmeasured.

Evidence: S07 docs/research_v2/gap_closure.md, S08 experiments/research_v2/release_manifest.json

14. Security: implemented execution boundaries

The safety goal is concrete: model-generated code must not gain arbitrary access to the host, hidden test evidence, the network or unrestricted resources. The model is not trusted to enforce that boundary. Trusted host tools validate edits, and a capability-limited guest executes the program.

Risk	Implemented measure	Remaining boundary
Host compromise / file access	CPython WASI through Wasmtime; only /lib and disposable /task preopened read-only to generated code. Host tools apply edits.	Not a formal proof of runtime security; trusted host, runtime and curated library remain in the trusted computing base.
Exfiltration / subprocesses	No guest network or unrelated filesystem capability; documented file/socket/process denial probes.	No claim of comprehensive adversarial red-team coverage or universal prompt-injection resistance.
Runaway execution	Guest memory cap 256 MiB, fuel 5 billion; host CPU limit 6/7 seconds and output-file limit 8 MiB; host-side deadlines.	These are source defaults, not proof every deployment is configured identically.
Unsafe paths / edits	Reject absolute and traversal paths; typed tool envelopes, bounded edits and unique-patch operations.	Trusted host path validation and symlink behavior deserve continuing adversarial testing.
Hidden-evidence leakage	Private tests and correct sources kept outside learner view; terminal-only private assessment.	External pretraining exposure and all possible semantic overlap remain unknown.
False completion	Independent value-and-type comparison, terminal tests and complete replay/hash audit.	Replay shares parts of transition logic; shared implementation defects remain possible.

Native macOS sandbox creation was denied and Docker/Podman were absent. The project did not respond by running untrusted code in host Python; it used WASI and failed closed when a runtime could not be enforced. The SQLite secondary environment restricts execution to SELECT-style capabilities, limits rows/time and denies extensions/file access.

Evidence: S13 src/mindscape/coding/wasi_worker.py, S10 docs/coding/protocol.md, S09 docs/coding/final_architecture.md

15. Safe development and scientific integrity

• Frozen data/protocol before evaluation: explicit task identities, nested training budgets, seeds and model revisions reduce hidden tuning and duplicate counting.

• Separate training-only teacher evidence from evaluation observations; do not turn successful evaluation repairs into verified training memory.

• Retain actual failed runs, unsuccessful trajectories, malformed proposals, operational failures and old negative experiments. Missing measurements remain unknown, never zero.

• Preserve source/history and immutable releases. Normal Git pushes, checksum-bound archives and clean-destination restoration avoid silent replacement of old evidence.

• Incremental per-episode saving and deduplication by frozen keys allow resume without starting a competing worker or counting partial rows as finished.

• Validate model/adapter/source/data hashes, byte-identical original prefixes, raw-to-statistic agreement and full replay before claiming coverage completion.

• Record CPU/MPS precision and batching strata, individual latency versus throughput, diagnostic overhead and unknown cost separately.

• Keep credentials outside public source and command output. The public website exposes bounded research artifacts, no secret model credentials; its static CSP and security headers reduce presentation-layer risk.

V1 acceptance included 130 tests plus six subtests in original and clean-restored environments, scoped lint, compilation, wheel build, fairness checks and 6,242-file snapshot verification. V2 numeric cloud validation recorded 137 tests plus six subtests; scoped closure recorded 138 plus six, and two focused observer tests. These are historically evidenced runs, not freshly rerun counts for this report.

Security is not equivalent to model alignment. The current measures bound execution and evidence access; there is no measured general harmful-content rate, semantic hallucination detector, production abuse-monitoring service, formal security certification, or comprehensive prompt-injection benchmark. The 172 failed coding answers are terminally incorrect programs, not a measured semantic hallucination rate.

Evidence: S14 docs/final_package_verification.md, S02 docs/final_research_report.md, S06 docs/research_v2/scientific_report.md, S07 docs/research_v2/gap_closure.md

16. Where we stand and the next justified work

Verified: a completed narrow coding study; clean restoration and preserved release; explicit execution/replay infrastructure; substantial negative evidence; controlled numeric masking findings; targeted task-variant controls; real model-forward coding telemetry; and a deployed evidence website. Full scientific competitiveness is not established.

Not supported: broad GPT/Gemini superiority, a causal architecture-only coding advantage, a threshold-crossing data-efficiency ratio, usefulness of dreaming, independently replicated cross-domain learned cognition, multimodal capability, or a learned world model. Strong custom IID results do not erase weaker OOD behavior.

Smallest useful research priorities - recommendations, not executed work

• Freeze a new confirmatory masking study whose legal-action choices are genuinely non-singleton, while preserving scoring and explicitly distinguishing controller intervention from task difficulty.

• Use independent implementation/training seeds and new held-out task families to test whether the controller finding and learned-policy effects generalize.

• Run matched-information and matched-interaction coding controls so any architecture contribution can be separated from teacher supervision, extra tokens and repeated edits.

• Extend live telemetry to representative additional adapters/conditions; do not fabricate unavailable historical activations.

• Investigate OOD failure groups before expanding expensive sweeps. Preserve an untouched final evaluation set during development.

• Define exact historical model releases, shared tasks/prompts/scoring and an authorized API budget before running proprietary references.

• Add targeted adversarial runtime/path/leakage tests and independent transition verification where shared implementation risk matters.

No neural training, benchmark rerun, paid API request, model download, new release tag or website redesign was performed in preparing this package. User-led redesign can reuse the PDF graphs and editable SVGs without altering the research evidence.

Evidence: S15 docs/final_limitations.md, S07 docs/research_v2/gap_closure.md, S08 experiments/research_v2/release_manifest.json

17. Reproduction and provenance

Repository: https://github.com/akilanpl/mindscape . This report reads checkout 33c5b5657b2d19741b7b0ac84388d4f7997f9582. Research v1 is preserved at 2862f5ae54443b231abda4255bdd6c652eca2343 with tag mindscape-research-complete-v1. Scientific closure is bound to c530faa and its earlier protocol/execution commits. No Research v2 completion tag is asserted.

The numeric export has 3,876 files and 11,957,855 compressed bytes. The scoped closure archive has 328 restored files, 2,417,409 compressed bytes and SHA-256 d19d18616d32f9360c1226da3f423b7cf0a94d741f898acbf4804cf88d8b8259. Counts differ because each package has a different scope; they must not be summed as a single unique-file inventory.

Use the pinned archive restoration and verification commands in docs/research_v2/reproduction.md and gap_closure.md. Saved-evidence verification does not require neural inference. Live diagnostics require the existing checksum-pinned weights. The numeric clean restoration and the coding full snapshot restoration are separate processes.

Evidence ledger

S01 | README.md
SHA-256 68c79e799f45d0c773bbfa6d894d1ec231d0aaccd78ec70e56d4e01a4324c127

Open pinned repository source

S02 | docs/final_research_report.md
SHA-256 47a97c4584077f2b2ab4f8dfce02d79a9b7a8c7879ae8a3a765b5911027e1f9f

Open pinned repository source

S03 | experiments/coding_completion_v2/final_summary.json
SHA-256 ae36610fb3f82c42e6c70a32325baa58676a9b6c1f6a4b5493e83ddec20ddddd

Open pinned repository source

S04 | docs/final_results.md
SHA-256 2d1fb3f1b3787048732c83d921996d31938eb305d7a317c1db941b299388c117

Open pinned repository source

S05 | experiments/final_smollm2_v1/aggregate.json
SHA-256 48e4e14d80f4b578f526e4e51617429ce293beff977739241f5d128f43d073b0

Open pinned repository source

S06 | docs/research_v2/scientific_report.md
SHA-256 eb265d0d82c7916d5998f45beaac8d69d6b6a23818e2230e469625a441ddc618

Open pinned repository source

S07 | docs/research_v2/gap_closure.md
SHA-256 e6e86a95a379e05a6ff00f83e4031e9a675f923482b78cfd0b50699f7401f0ba

Open pinned repository source

S08 | experiments/research_v2/release_manifest.json
SHA-256 5c9362598d2104bfb59c061a4a0c0eb0e17f5e8d84f63129e507b15d0a591a78

Open pinned repository source

S09 | docs/coding/final_architecture.md
SHA-256 ee9caa86f16b82a8f342b15419b9771c8e25807e2575d23d9e32336eb7137b22

Open pinned repository source

S10 | docs/coding/protocol.md
SHA-256 09ea8d0dcd37a1d22d8858988708df574a32b705dd14ca5277c1d4d28efcf397

Open pinned repository source

S11 | docs/coding/historical_gpt_gemini_comparison.md
SHA-256 847036c54c1453c6742a67713aef6b8333752e28f1a92bb155616c19265ba5f1

Open pinned repository source

S12 | docs/historical_references.md
SHA-256 3b7c276bfa57e997a135184629a1ea0298535dcb3fd9778e0eefd806d64ac3d9

Open pinned repository source

S13 | src/mindscape/coding/wasi_worker.py
SHA-256 9c97136c65db77dcb7aa854fb220341d4ba0cbd59630a6861cf4c19388f5c790

Open pinned repository source

S14 | docs/final_package_verification.md
SHA-256 2bb8db7ae7c7a9fbed1dbbaf8f7c30a2e8e83354312b80458fe02734ac8e7b22

Open pinned repository source

S15 | docs/final_limitations.md
SHA-256 7dab8d2c6777285acb3b1f5e424146ff118db68e590c65fbc06a7deb1565753d

Open pinned repository source

S16 | docs/research_v2/reproduction.md
SHA-256 ab22f99bef1a93e99659698bef1bdbfa8b4e9eea92847d87d0566d403585e51a

Open pinned repository source

External context sources: OpenAI GPT-4 Technical Report, Table 2 (March 2023); Gemini report arXiv 2312.11805v1, Table 2 (December 2023). Historical exact API checkpoint/sampling details remain unknown; direct matched Mindscape comparisons are still unrun.